Public Key Cryptography

- New paradigm introduced by Diffie and Hellman
- The mailbox analogy:
  - Bob has a locked mailbox
  - Alice can insert a letter into the box, but can't unlock it to take mail out
  - Bob has the key and can take mail out
- Encrypt messages to Bob with Bob's public key
  - Can freely distribute
- Bob decrypts his messages with his private key
  - Only Bob knows this

Requirements

- How should a public key scheme work?
- Three main conditions
  - It must be computationally easy to encrypt or decrypt a message given the appropriate key
  - It must be computationally infeasible to derive the private key from the public key
  - It must be computationally infeasible to determine the private key from chosen plaintext attack
    - Attacker can pick any message, have it encrypted, and obtain the ciphertext

Hard problems

- Public key cryptosystems are based on hard problems
- Discrete Logarithm Problem (DLP)

- Given:
  - Multiplicative group G
  - Element a in G
  - Output b
- Find:
  - Unique solution to $a^x = b$ in G
    - x is $\log_a b$

- No polynomial time algorithm exists to solve this*

*On classical computers

Diffie Hellman Key Exchange

- Developed by Martin Hellman, Ralph Merkle, and Whitfield Diffie at Stanford University in 1976
- All users share common modulus, p, and element g
  - $g \neq 0$, $g \neq 1$, and $g \neq p-1$
- Alice chooses her private key, k_A
  - Computes $K_A = g^{k_A} \bmod p$ and sends it to Bob in the clear
- Bob chooses his private key, k_B
  - Computes $K_B = g^{k_B} \bmod p$ and sends it to Alice in the clear
- When Alice and Bob want to agree on a shared key, they compute a shared secret S
  - $S_{A,B} = K_B^{k_A} \bmod p$
  - $S_{B,A} = K_A^{k_B} \bmod p$

Why does DH work?

- $S_{A,B} = S_{B,A}$
- $(g^{k_A})^{k_B} \bmod p = (g^{k_B})^{k_A} \bmod p$

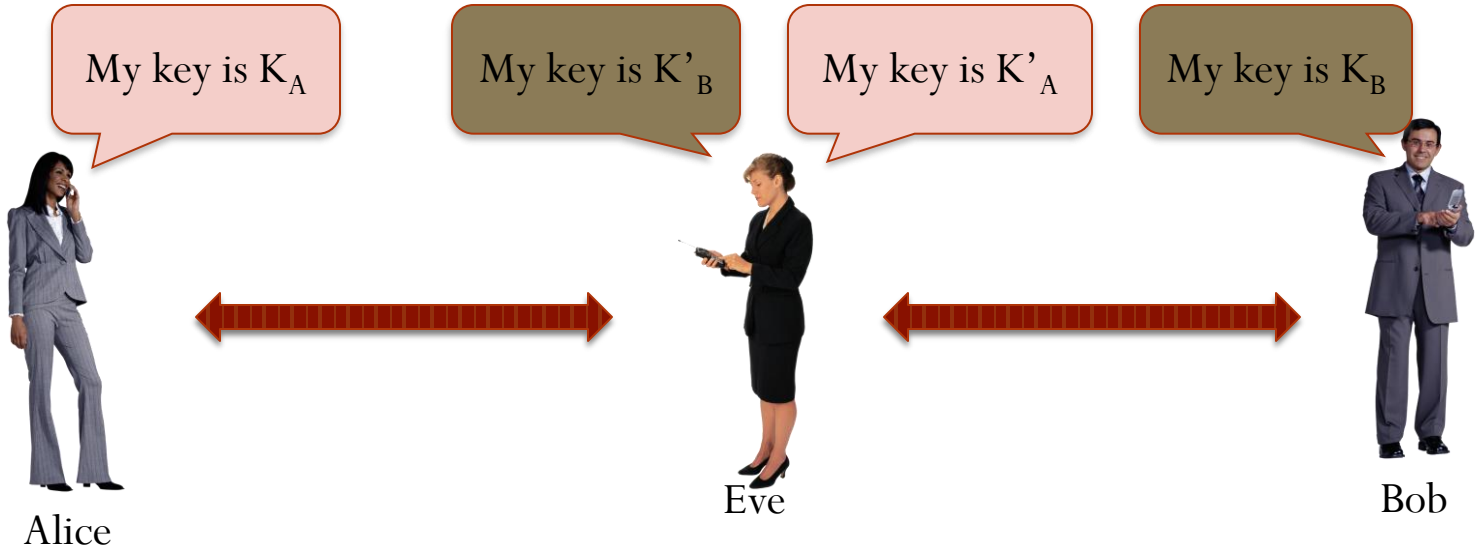
- Eve knows
  - g and p
  - K_A and K_B
  - Why can't Eve compute the secret?

$$S_{A,B} = K_B^{k_A} \bmod p$$
$$S_{B,A} = K_A^{k_B} \bmod p$$

- This was the first public key cryptography scheme

Could it fail?

- Eve could fool Alice and Bob
  - Man in the middle / bucket brigade



Alice has no guarantee that the person she's establishing a key with is actually Bob

RSA

- Rivest-Shamir-Adleman
- Probably the most well-known public key scheme
- First, some background

Euler's Totient

- Totient function $\phi(n)$
  - Number of positive numbers less than n that are relatively prime to n
    - Two numbers are relatively prime when their greatest common divisor is 1

- Example: $\phi(10) = 4$
  - 1, 3, 7, 9

- Example: $\phi(7) = 6$
  - 1, 2, 3, 4, 5, 6
  - If n is prime, $\phi(n) = n-1$

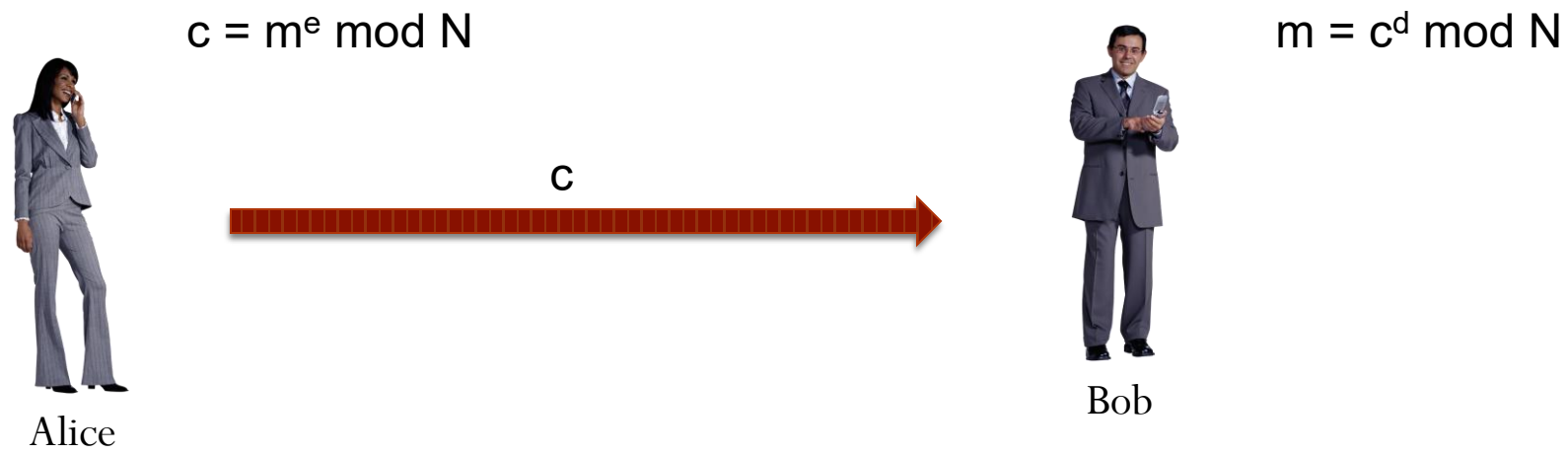
RSA keys

- Choose 2 large primes, p and q
- $N = pq$
- $\phi(N) = (p-1)(q-1)$
- Choose $e < N$ such that $\gcd(e, \phi(N))=1$
- d such that $ed = 1 \bmod \phi(N)$

- Public key: {N, e}
- Private key: {d}
  - p and q must also be kept secret

RSA encryption/decryption

- Alice wants to send Bob message m
  - She knows his public key, {N,e}



Toy example

- p=7, q=11
  - N=77
  - $\phi(N) = (6)(10) = 60$
- Bob chooses e=17
  - Uses extended Euclidean algorithm to find inverse of e mod 60
  - Finds d=53
- Bob makes {N, e} public

Toy example (continued)

- Alice wants to send Bob "HELLO WORLD"
- Represent each letter as a number 00(A) to 25(Z)
  - 26 is a space
- Calculates:
  - $07^{17} \bmod 77 = 28$, $04^{17} \bmod 77 = 16$, …, $03^{17} \bmod 77 = 75$
- Sends Bob 28 16 44 44 42 38 22 42 19 44 75
- He decrypts each number with his private key and gets "HELLO WORLD"

RSA Example

- Find two large prime integers, p and q, and form product $n = pq$
- Find a random integer, e, that is relatively prime to $\Phi(n) = (p-1)(q-1)$
- p and q are kept private, (n,e) are the public key
- Message is partitioned into blocks, b, such that $b < n$
- Each block is encrypted using the equation: $c = b^e \bmod n$
- For the private key, calculate integer d which is the modular inverse of e for $\Phi(n)$, or $e * d \bmod \Phi(n) = 1$
- Once d is calculated it becomes your private key and all records of p and q should be destroyed
- Each encrypted block, c, is decrypted using the equation: $b = c^d \bmod n$
- $p = 61$, $q = 53$, $n = 3233$, $\Phi(n) = 3120$, $e = 17$, $d = 2753$
- encrypt(123) $= 123^{17} \bmod 3233 = 855$
- decrypt(855) $= 855^{2753} \bmod 3233 = 123$

What could go wrong?

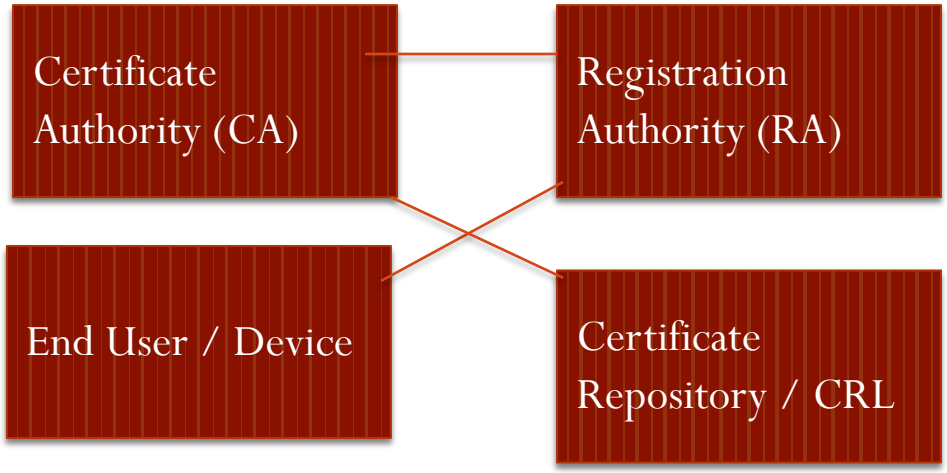
- What was wrong with the toy example?
  - Eve can easily find the encryption of each letter and use that as a key to Alice's message
  - Even without knowing the public key, can use statistics to find likely messages
    - Like cryptogram puzzles

How it should really happen

- p and q should be at least 512 bits each
  - N at least 1024 bits
- The message "HELLO WORLD" would be converted into one very large integer
- That integer would be raised to the public/private exponent
- For short message, pad them with a random string

Is this key yours?

- How to bind a key to an identity?
  - PKI
    - Framework for managing digital keys and certificates
    - Enables secure exchange of information over untrusted networks
    - Provides authentication, confidentiality, integrity, and non-repudiation



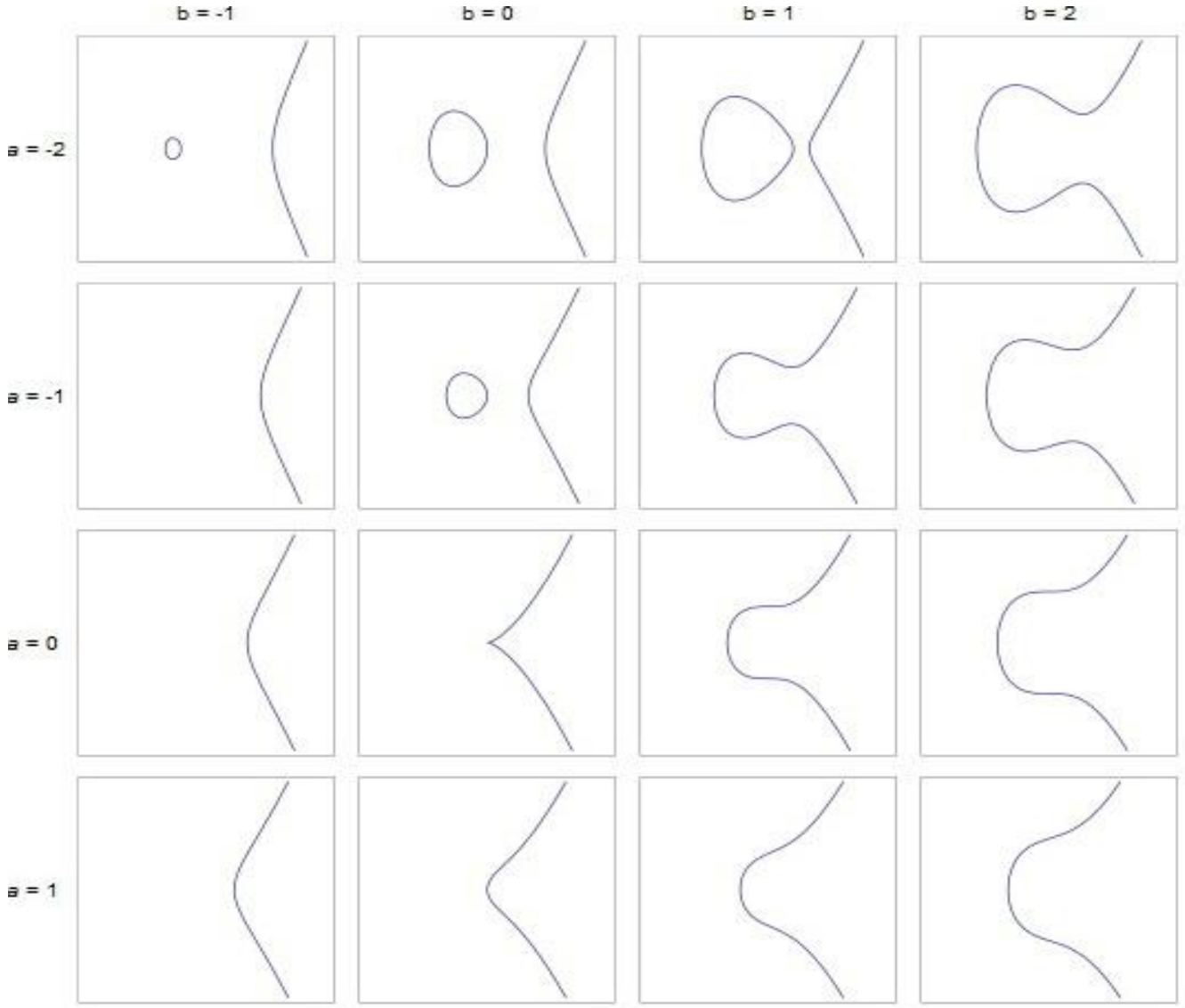
- Hierarchical trust model
- Single-root CA/Multi-root CA
- Web of Trust
- Revocation inefficiencies

PK Paradigm

- Genkey(some info)
  - Creates K_{pub} and K_{priv}
- Encrypt with K_{pub}
- Decrypt with K_{priv}
- Certificate binds key to individual

More Advanced

- What are Elliptic Curves?
  - Curve with standard form $y^2 = x^3 + ax + b \quad a, b \in \mathbb{R}$
- Characteristics of Elliptic Curve
  - Forms an abelian group
  - Symmetric about the x-axis
  - *Point at Infinity* acting as the identity element



Examples of Elliptic Curves

Finite Fields

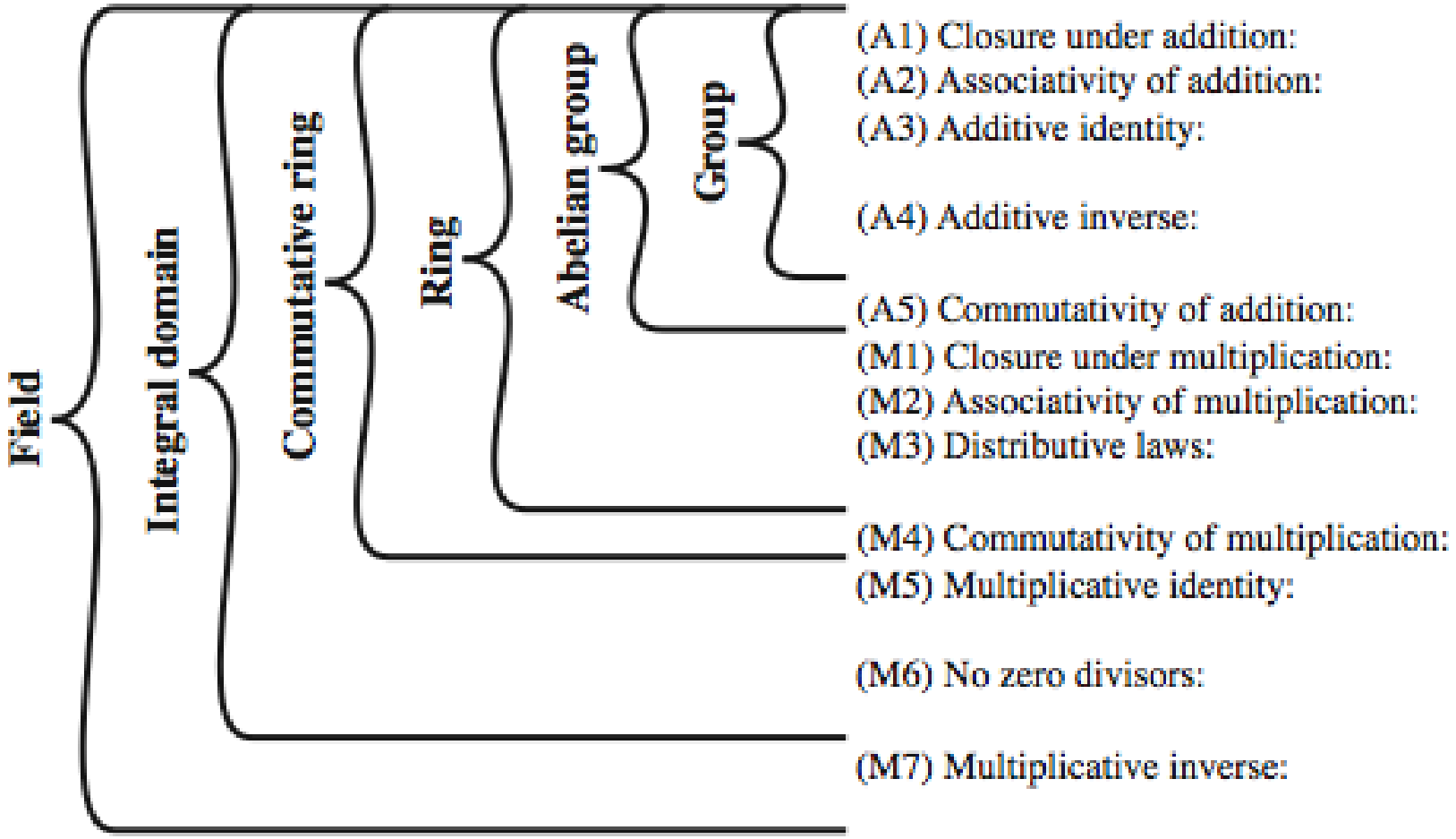
- aka Galois Field

- $GF(p^n)$ = a set of integers $\{0, 1, 2, \ldots, p^n - 1)$

  where p is a prime, n is a positive integer

- It is denoted by $\{F, +, x\}$

  where + and x are the group operators



Group, Ring, Field

Why Elliptic Curve Cryptography?

- Shorter Key Length
- Lesser Computational Complexity
- Low Power Requirement
- More Secure

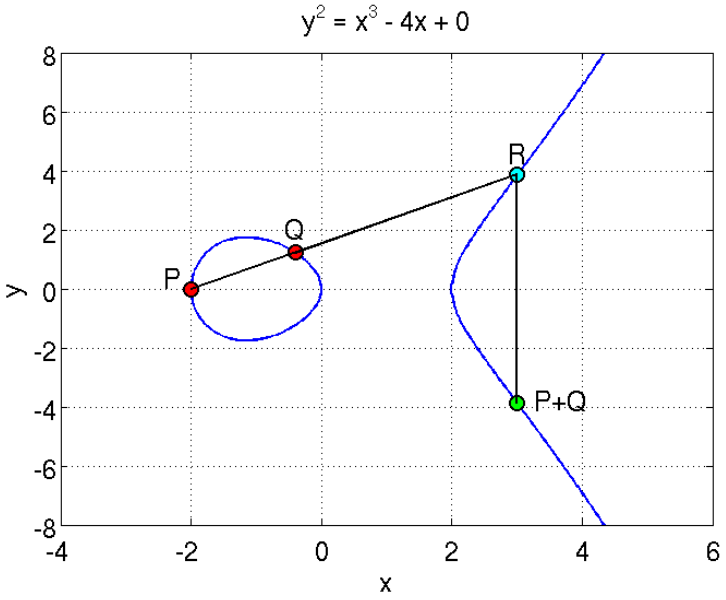
Comparable Key Sizes for Equivalent Security

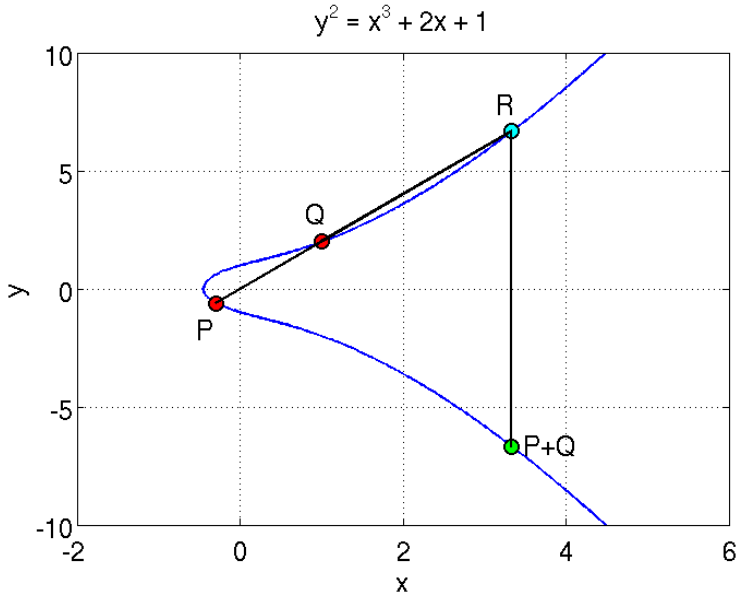
Symmetric Encryption (Key Size in bits)	RSA and Diffie-Hellman (modulus size in bits)	ECC Key Size in bits
56	512	112
80	1024	160
112	2048	224
128	3072	256
192	7680	384
256	15360	512

What is Elliptic Curve Cryptography?

- Implementing Group Operations
  - Main operations - point addition and point multiplication
  - Adding two points that lie on an Elliptic Curve – results in a third point on the curve
  - Point multiplication is repeated addition
  - If P is a known point on the curve (aka Base point; part of domain parameters) and it is multiplied by a scalar k, Q=kP is the operation of adding P + P + P + P… +P (k times)
  - Q is the resulting public key and k is the private key in the public-private key pair

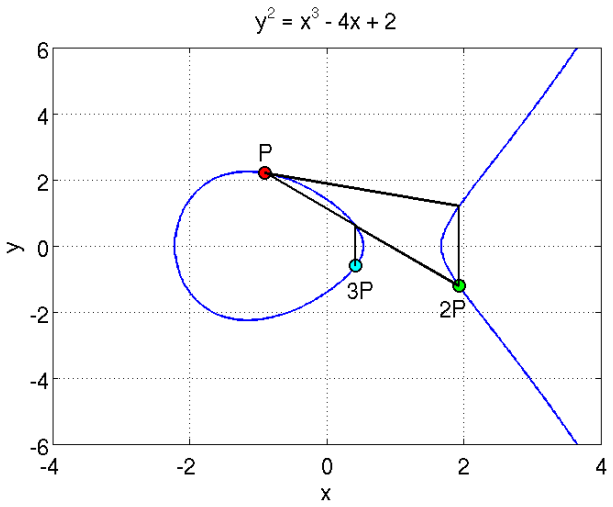
What is Elliptic Curve Cryptography?

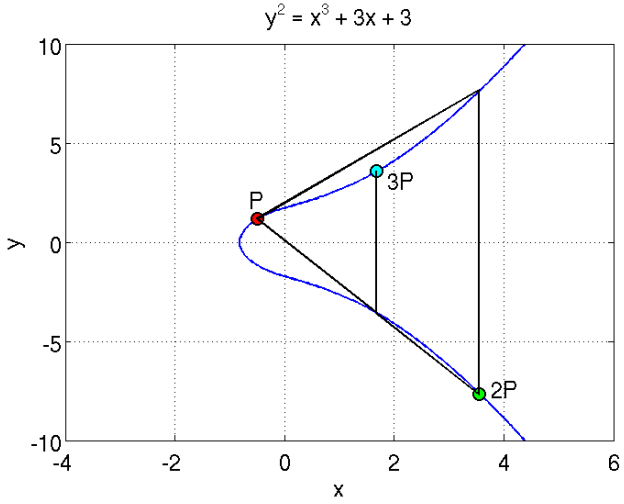




- Adding two points on the curve
- P and Q are added to obtain P+Q which is a reflection of R along the X axis

What is Elliptic Curve Cryptography?





- A tangent at P is extended to cut the curve at a point; its reflection is 2P
- Adding P and 2P gives 3P
- Similarly, such operations can be performed as many times as desired to obtain Q = kP

What is Elliptic Curve Cryptography?

- Discrete Log Problem
  - The security of ECC is due the intractability or difficulty of solving the inverse operation of finding k given Q and P
  - This is termed as the discrete log problem
  - Methods to solve include brute force and Pollard's Rho attack both of which are computationally expensive or unfeasible
  - The version applicable in ECC is called the Elliptic Curve Discrete Log Problem
  - Exponential running time